

Week 3 / 6

Git 常用操作 Part II

Git operations part2

GitHub for UI Designers

Karen Shen

May19, 2026

AGENDA

這週會講什麼

What we'll cover today

A

Git 名詞釐清

Vocabulary Quick Reference

- Commit vs Push
- Branch vs Fork + 何時開 branch

B

Changes 管理

Managing Changes — Quick Review

- Staged / Unstaged / Discard / Stash
- Agent Session 保存
- PR Review 流程 + 原則

C

三種 Working Repo

Three Working Repo Types — Main Focus

- Type A 協作 repo
- Type B 引用 repo (DS + 部門規範)
- Type C 個人 repo

Commit vs Push

What's the difference?

	Commit	Push
做了什麼 Action	在本機建立存檔點	把存檔點上傳到 GitHub
誰看得到 Visibility	只有你	團隊所有人
可以反悔嗎 Reversible?	可以，push 之前都在你電腦裡	Push 後別人可能已經看到
Figma 類比 Figma analogy	存檔 (Cmd+S)	分享連結給同事



可以 commit 很多次再一起 push ; Push 的時機：收工、換裝置、同步到 remote repo

Branch vs Fork

And when to use which

	Branch	Fork
在哪裡	同一個 repo 裡	複製到你自己的帳號
誰用	有寫入權限的人	外部貢獻者
Figma 類比	Figma Branch	Duplicate 整個檔到 Draft

怎麼判斷？

你是 repo 成員？

Branch

改別人的開源專案？

Fork

拿當模板？

Fork

常見情境

改公司 repo 自己負責的頁面

Branch

幫開源 design system 修 icon

Fork

personal repo 做實驗

Branch

什麼時候開 Branch？怎麼開？

When and how to create a branch

原則：編輯開始前就開，不是 commit 的時候才開



💡 忘了開就直接在 main 改了？只要還沒 commit，現在開 branch，改動會自動跟過去

三種開 Branch 的方式

GitHub 網頁	Branch 切換器 → 輸入新名稱 → Enter	小改動、改文件，不需要本機跑程式	可以
對 Agent 說	「幫我開一條 branch 叫 feat/xxx」	用 Claude Code 工作時（最常用）	最推薦
指令	git checkout -b feat/xxx	自己熟悉指令，想手動控制	進階

查看 Changes + Discard

Viewing and reverting your changes

S **Staged**
已選好要 commit 的改動

M **Unstaged**
改了但還沒決定要不要存

U **Untracked**
全新檔案, Git 還不認識

用 Claude Code 時通常不用手動管理 — Agent 會自動判斷該 commit 哪些檔

Discard Changes — 還原不想要的改動

放棄單一檔案

「把 Header.tsx 還原到上次 commit 的狀態」

安全

放棄所有改動

「放棄所有未 commit 的改動」

⚠ 慎

只是想看看上一版

「顯示 Header.tsx 上次 commit 的內容」

安全

⚠ Discard 不可逆 — 改動會真的消失。不確定就先 commit 或 stash

Stash：暫存但不 commit

Save work-in-progress without committing

改到一半要切任務？先把改動「收起來」，之後再拿出來繼續。

要切 branch，但目前改動還沒好

「先 stash 目前的改動」

處理完緊急事，想繼續剛才的工作

「把 stash 的東西拿回來」

看看 stash 裡有什麼

「列出目前的 stash」

stash 的東西不要了

「清掉 stash」

Stash vs Discard

Stash

改動保留，收起來，隨時拿回來

Discard

改動消失，⚠️ 無法復原

Agent Session 怎麼保存？

Saving code vs. agent context when you pause

重開機或明天繼續時，Code 的狀態跟 Agent 的記憶是兩件事，要分開處理。



Code 的狀態

Git 負責

- 改好了 → commit + push
- 改到一半不想 commit
→ git stash (本機)
→ WIP commit (需跨裝置時)



Agent 的記憶

對話 context — 要主動保存

- Session 關掉就消失
下次開啟 agent 是「失憶」的
- 暫停前：叫 agent 把進度、
下一步寫成一份 handoff note
- 下次開工：讓 agent 讀那份 note
接回來繼續

Code 靠 Git 保存

Agent 記憶靠你叫它「先寫下來」

Merge to main

Sharing your work — Pull Request review

個人 commit 完，怎麼把改動交給別人看、討論、合進 main

PR Review 流程

Open PR → Review → Merge

從個人分支進到團隊 main，中間是一段協作儀式 — 兩個角色都會碰到。



As a Requester

讓 reviewer 用最少力氣看懂

- ☐ PR 範圍小 — 一個 PR 只解一件事
- ☐ Description 三段：what / why / how to test
- ☐ 貼 before/after 截圖或預覽連結
- ☐ Tag reviewer
- ☐ 回應每條 comment（修了 / 不同意 / 之後）



As a Reviewer

提問為主，幫作者把盲點找出來

- ☐ 先讀 description，掃 PR scope
- ☐ 只 review 改動的範圍，不順手碰整檔
- ☐ 對照 before/after
- ☐ 提問優先於下決定（「這是 DS 嗎？」）
- ☐ 區分 violation（必修）vs 偏好（討論）

把對方需要的資訊準備好，review 才不會變成猜謎或拉鋸戰

Merge Conflict — 怎麼發生、怎麼處理

Why conflicts happen & how to handle them

Git 沒法自動合併兩條 branch 對同一段 code 的不同改動 — 需要人來判斷。

⚠ 為什麼會發生

Common causes

1. Branch 開太久沒同步 main

main 已經往前走、你的 branch 還停在分岔點

2. 兩條 branch 改到同一段 code

你跟同事改了同一個 component / 同一行 css

3. 同檔多人同時編輯

多個設計師動同一個 mockup html



原則上怎麼處理

Handling principles

- 開 PR 前先 git pull main + rebase (或 merge)
- Conflict 出現先看清楚兩邊內容、不要讓Agent決定
- 不確定該保哪邊 → 找另一位 author 一起確認
- 解完跑一次 build / 預覽 → 確認沒漏改

衝突不是錯誤，是 git 把判斷權交還給人

Upcoming - Design PR review skill

design-pr-review — a Claude Code skill

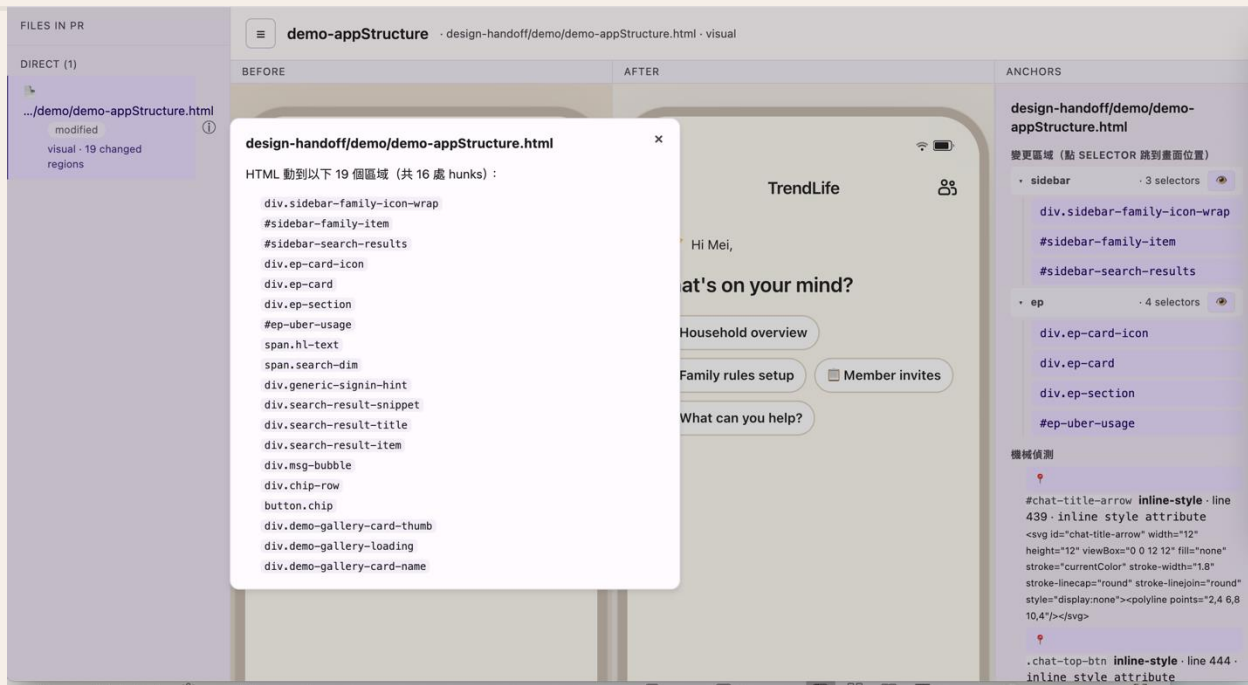
視覺化比對差異，協助決策



Skill

- 自動 scope PR (只看改動的檔案)
- Before / After side-by-side (mockup 視覺差異)
- Design-system violation 以「提問」方式 flag 出來
- 多人改的衝突檔案掃描 (同檔多 author → 標出來)
- Draft 友善 tone 的留言 交給 pr-comment 發佈

PR #259



Trigger : 「review this design PR」 / 「設計 PR review」 / design-pr-review (PR# or link)

三種 Working Repo

Three Working Repo Types

Type A 協作 repo · Type B 引用 repo · Type C 個人 repo

你跟這個 repo 的關係是什麼？

What's your relationship with this repo?

A

協作 repo

Collaboration

典型例子

產品 repo、team repo

我的角色

貢獻者（有寫入權限）

核心機制

Branch → PR → Merge

改動頻率

每天都在改

B

引用 repo

Reference

典型例子

Design System、部門規範, PM requirements

我的角色

讀者（很少編輯）

核心機制

Agent hooks + local cache

改動頻率

跟著版本更新

C

個人 repo

Personal

典型例子

Side project、實驗、dotfiles

我的角色

擁有者（完全自主）

核心機制

Commit + Push（無 PR）

改動頻率

自己決定

你是貢獻者

You have write access — work with your team

核心觀念

- Main branch = 團隊共識版本，不直接改
- 你的工作在 branch 上進行
- 改好了 → PR → review → merge 才進 main
- Agent 幫你 commit + push，你決定 PR 時機

日常同步流程

開工前



pull main, 更新到最新

工作中



branch 上 commit, 視情況把 main merge 進來

收工前



push branch, 視情況開 PR

該注意什麼 + 怎麼除錯

What to watch out for + common fixes

該注意什麼

- 1 Pull 前先 commit 或 stash — 否則本機改動可能跟遠端衝突
- 2 確認你在對的 branch — 不要在 main 上直接改
- 3 PR 開了之後若 main 繼續有改動，要 sync（GitHub 的「Update branch」）

怎麼除錯

rejected (non-fast-forward)

→ 先 pull，再 push

CONFLICT

→ 手動解，或對 agent 說「幫我解這個 conflict」

不確定在哪個 branch

→ 問 agent「我現在在哪個 branch？」或看 VS Code 左下角

你是「讀者」，不是「貢獻者」

You consume this repo — rarely edit it

這個 repo 存在的目的：讓你的 Agent 知道規範是什麼，才能幫你設計出符合標準的東西

Type B 包含兩種內容，處理方式相同



Design System

- 設計 token（色票名稱、spacing、字型規格）
- 元件使用規則（什麼時候用哪個 variant）
- Do's & Don'ts



部門規範 / Conventions

- Design conventions、決策記錄
- Template、toolkit 文件
- Brand voice、用字規範

Agent 讀這些內容 → 才能在幫你寫 code 或生成設計時，自動遵守你的 DS 規範與部門標準

Agent 怎麼拿到這些內容？

How does your agent access the reference repo?



✅ 自動執行 — 平常你不需要做任何事

📅 TTL weekly — DS 改動慢，週更一次夠用

⚙️ 設定一次（通常由 department 統一配置），之後使用 hooks 自動維護

你需要做什麼？

Your responsibilities with a reference repo

自動

平常

Normal operation

什麼都不用做

Hooks 自動幫你維持 local cache 最新

偶爾

發現規範過時

Cache looks stale

手動觸發一次 fetch

確認 cron-job 有在跑，或手動 pull Department repo

少見

要回饋修正

Want to fix or update the convention

切換成 Type A 流程

到 Department repo 開 branch → 改 → PR，不要改 local cache



不要直接改 local cache 裡的內容 — 下次 fetch 會被覆蓋，改了等於沒改

完全自主 — 規則自己定

Your repo, your rules

設置 & 日常流程

1

建 GitHub repo + .gitignore

.env / node_modules / .DS_Store

2

Commit + Push (沒有 PR 壓力)

建議：每次工作結束 push 一次

3

跨裝置：git pull 拿最新

換電腦後先 pull，再開始工作

該注意什麼



不要 commit .env (API key、密碼)



Public repo 特別小心，不要放任何內部資料



Commit message 要讓三個月後的自己看得懂

發現 .env 已經 commit 了？→ 對 agent 說「幫我把 .env 從 git 記錄裡移除」

遇到一個 repo，你應該用哪種方式對待它？

Which type applies to this repo?

這個 repo 我需要改東西、跟人協作？

Do I need to edit and collaborate?



Type A 協作 repo

Branch → PR → Merge

這個 repo 我只是要讀它的規範或文件？

Do I just need to read it for reference?



Type B 引用 repo

Agent hooks + local cache

這個 repo 完全是我自己的、沒有協作壓力？

Is this purely mine — no team, no review?



Type C 個人 repo

Commit + Push, 自己決定節奏



同一個 repo 可能兩種都有 — 平常你是 Type B 讀者，偶爾有 feedback 時暫時切換成 Type A 貢獻者

本週三件事

Three things to remember

1

Commit = 存檔 ; Push = 分享。Branch 在任務開始前開。

Commit saves locally. Push shares. Always branch before you start editing.

2

三種 Repo, 三種對待方式 — 你的角色決定你怎麼做

Collaboration = branch+PR / Reference = agent reads / Personal = just push

3

Type B 的 DS 與部門規範, 靠 Agent hooks 自動同步 — 你不需要手動管

Reference repos are auto-synced via hooks. You only act when cache looks stale.

Additional discussion

Vibe coding is self-driving, not no driving

Keep your eyes on the road, even when AI takes the wheel.

